

Section A — Concept Application.

S1: Constraints.

- restaurants.restaurant_id is a PRIMARY KEY, so every restaurant has one unique, non-null identity. Without it you get duplicate restaurants, and orders can't reliably point to the right one.
- restaurants.name is NOT NULL. Without it you could save nameless restaurants that show up as blanks in the app.
- menu_items.restaurant_id is a FOREIGN KEY (and NOT NULL) referencing restaurants. Without it you get orphan menu items belonging to restaurants that don't exist, and you can delete a restaurant while its items are still in use.

S2: WHERE in UPDATE/DELETE.

- WHERE limits the change to the intended rows. For example, UPDATE orders SET status='Cancelled' WHERE status='Pending' AND order_date < NOW() - INTERVAL 7 DAY;.
- For the delete, use DELETE FROM menu_items WHERE status='Unavailable' AND restaurant_id IN (SELECT restaurant_id FROM restaurants WHERE is_closed=1);.
- Without WHERE, every row is changed or deleted. That means all orders cancelled, including delivered ones, or the whole menu wiped. It can also fail on foreign key errors or cause permanent data loss with no undo once committed. Best practice is to run a SELECT with the same WHERE first, and use a transaction.

S3: Revenue report.

- Use SUM(), AVG() and COUNT() with a JOIN, WHERE on the current month, GROUP BY restaurant_id, and HAVING COUNT(*) > 20.
- WHERE filters individual rows before grouping, so aggregates like COUNT(*) don't exist yet at that point. HAVING filters after grouping, so it can use aggregate results.

S4: INNER vs LEFT JOIN.

- INNER JOIN returns only rows that match in both tables. Orders whose restaurant no longer exists disappear from the result.
- LEFT JOIN (orders LEFT JOIN restaurants) keeps every order. Where there is no matching restaurant, the restaurant columns are NULL.
- Use LEFT JOIN, because the page must show every order.

Assessment
M5 - Database Management

S5: VIEW.

- A VIEW hides ID columns and joins, gives the front end one simple, stable object to query, and can limit access to sensitive columns. If the underlying tables change, only the view definition needs updating.
- Two limitations: it is generally read-only (a view built on joins and aggregates can't be reliably INSERTed into or UPDATEd), and it isn't stored data, so it re-runs the query each time and can be slower on big tables. Other valid limitations are that the view breaks if the underlying tables change, and it can't be indexed directly.

S6: Transactions.

- The mechanism is a **transaction**, which gives atomicity: all or nothing. Run START TRANSACTION, then the wallet UPDATE, then the payment INSERT.
- If both succeed, COMMIT makes them permanent. If either fails, ROLLBACK undoes everything, so the balance goes back to what it was.
- A SAVEPOINT marks a point inside the transaction, and ROLLBACK TO savepoint_name undoes only the work after it. Here it lets you keep an earlier step while retrying a later one.

--

=====

-- M5-A1 | Database Management | MySQL 8.x

-- Sections B (Tasks 1-4), C (Capstone) and D (AI-Augmented Learning)

-- Run each block separately and take a screenshot of every result grid.

--

=====

--

=====

-- SECTION B - TASK 1: Design and Populate a Restaurant Database

--

=====

CREATE DATABASE IF NOT EXISTS foodapp_db;

Assessment
M5 - Database Management

```
USE foodapp_db;
```

```
DROP TABLE IF EXISTS orders;    -- created in Task 2, dropped first so re-runs work
```

```
DROP TABLE IF EXISTS restaurants;
```

```
CREATE TABLE restaurants (  
    restaurant_id INT AUTO_INCREMENT PRIMARY KEY,  
    name    VARCHAR(100) NOT NULL,  
    city    VARCHAR(50),  
    cuisine_type VARCHAR(50),  
    rating  DECIMAL(3,1)  
);
```

```
-- 7 restaurants across 4 cities (one will be deleted below)
```

```
INSERT INTO restaurants (name, city, cuisine_type, rating) VALUES  
( 'Spice Villa',  'Rajkot',  'North Indian', 4.3),  
( 'Pizza Palace', 'Ahmedabad', 'Italian',    4.1),  
( 'Dragon Wok',   'Surat',   'Chinese',    3.9),  
( 'Dosa Corner',  'Rajkot',  'South Indian', 4.5),  
( 'Burger Hub',   'Ahmedabad', 'Fast Food',   3.6),  
( 'Sweet Tooth',  'Vadodara', 'Desserts',   4.0),  
( 'Old Tandoor',  'Surat',   'Mughlai',   3.2);
```

```
-- UPDATE with WHERE
```

```
UPDATE restaurants
```

```
SET rating = 4.6
```

```
WHERE name = 'Pizza Palace';
```

Assessment
M5 - Database Management

-- DELETE with WHERE

DELETE FROM restaurants

WHERE name = 'Old Tandoor';

-- SELECT sorted by rating (highest first)

SELECT * FROM restaurants

ORDER BY rating DESC;

--

=====

-- SECTION B - TASK 2: Aggregate Revenue Reporting on Orders

-- (uses the same foodapp_db.restaurants table from Task 1)

--

=====

CREATE TABLE orders (

 order_id INT AUTO_INCREMENT PRIMARY KEY,

 restaurant_id INT NOT NULL,

 customer_name VARCHAR(100) NOT NULL,

 total_amount DECIMAL(10,2) NOT NULL,

 order_date DATE NOT NULL,

 FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)

);

-- 14 orders across 4 restaurants (ids 1-4)

INSERT INTO orders (restaurant_id, customer_name, total_amount, order_date) VALUES

(1, 'Amit', 1200.00, '2026-09-01'),

(1, 'Riya', 950.00, '2026-09-04'),

(1, 'Karan', 1800.00, '2026-09-09'),

Assessment
M5 - Database Management

(1, 'Neha', 1500.00, '2026-09-15'),
(2, 'Priya', 2200.00, '2026-09-02'),
(2, 'Rahul', 1750.00, '2026-09-07'),
(2, 'Sneha', 1300.00, '2026-09-12'),
(3, 'Vikram', 600.00, '2026-09-03'),
(3, 'Amit', 850.00, '2026-09-10'),
(3, 'Riya', 700.00, '2026-09-18'),
(4, 'Karan', 1100.00, '2026-09-05'),
(4, 'Neha', 900.00, '2026-09-11'),
(4, 'Priya', 1400.00, '2026-09-17'),
(4, 'Rahul', 500.00, '2026-09-20');

-- (a) Total revenue and average order value per restaurant

```
SELECT r.name,  
       COUNT(o.order_id)    AS total_orders,  
       SUM(o.total_amount)  AS total_revenue,  
       ROUND(AVG(o.total_amount),2) AS avg_order_value  
FROM orders o  
JOIN restaurants r ON r.restaurant_id = o.restaurant_id  
GROUP BY r.restaurant_id, r.name;
```

-- (b) Only restaurants with total revenue > Rs 5,000

```
SELECT r.name,  
       SUM(o.total_amount) AS total_revenue  
FROM orders o  
JOIN restaurants r ON r.restaurant_id = o.restaurant_id  
GROUP BY r.restaurant_id, r.name  
HAVING SUM(o.total_amount) > 5000;
```

Assessment
M5 - Database Management

-- (c) 5 most recent orders

SELECT * FROM orders

ORDER BY order_date DESC, order_id DESC

LIMIT 5;

--

=====

-- SECTION B - TASK 3: Multi-Table JOINS and Menu Summary View

-- (separate database so it does not clash with Task 1's restaurants table)

--

=====

CREATE DATABASE IF NOT EXISTS menu_db;

USE menu_db;

DROP VIEW IF EXISTS restaurant_menu_summary;

DROP TABLE IF EXISTS menu_items;

DROP TABLE IF EXISTS restaurants;

CREATE TABLE restaurants (

 restaurant_id INT AUTO_INCREMENT PRIMARY KEY,

 name VARCHAR(100) NOT NULL,

 city VARCHAR(50)

);

CREATE TABLE menu_items (

 item_id INT AUTO_INCREMENT PRIMARY KEY,

 restaurant_id INT NOT NULL,

Assessment
M5 - Database Management

```
item_name  VARCHAR(100) NOT NULL,  
price      DECIMAL(8,2) NOT NULL,  
category   VARCHAR(50),  
FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)  
);
```

```
INSERT INTO restaurants (name, city) VALUES  
( 'Spice Villa', 'Rajkot'),  
( 'Pizza Palace', 'Ahmedabad'),  
( 'Dragon Wok', 'Surat'),  
( 'Dosa Corner', 'Rajkot'),  
( 'New Cafe', 'Vadodara');    -- restaurant 5 has NO menu items
```

```
INSERT INTO menu_items (restaurant_id, item_name, price, category) VALUES  
(1, 'Paneer Tikka', 220.00, 'Starter'),  
(1, 'Dal Makhani', 260.00, 'Main Course'),  
(1, 'Gulab Jamun', 90.00, 'Dessert'),  
(2, 'Garlic Bread', 150.00, 'Starter'),  
(2, 'Margherita Pizza', 320.00, 'Main Course'),  
(2, 'Tiramisu', 180.00, 'Dessert'),  
(3, 'Veg Spring Rolls', 170.00, 'Starter'),  
(3, 'Hakka Noodles', 240.00, 'Main Course'),  
(3, 'Lemon Iced Tea', 90.00, 'Beverage'),  
(4, 'Masala Dosa', 130.00, 'Main Course'),  
(4, 'Idli Sambar', 80.00, 'Starter'),  
(4, 'Filter Coffee', 50.00, 'Beverage');
```

-- INNER JOIN: only restaurants that have items

Assessment
M5 - Database Management

```
SELECT r.name AS restaurant_name, m.item_name, m.price
FROM restaurants r
INNER JOIN menu_items m ON m.restaurant_id = r.restaurant_id;
```

-- LEFT JOIN: restaurants with no items also appear (NULLs in item columns)

```
SELECT r.name AS restaurant_name, m.item_name, m.price
FROM restaurants r
LEFT JOIN menu_items m ON m.restaurant_id = r.restaurant_id;
```

-- VIEW: item count and average price per restaurant

```
CREATE VIEW restaurant_menu_summary AS
SELECT r.name          AS restaurant_name,
       COUNT(m.item_id) AS total_menu_items,
       ROUND(AVG(m.price), 2) AS avg_item_price
FROM restaurants r
LEFT JOIN menu_items m ON m.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_id, r.name;
```

```
SELECT * FROM restaurant_menu_summary;
```

--

=====

-- SECTION B - TASK 4: Order Placement Procedure with Transaction Control

-- (MySQL stored procedure - MySQL has no PL/SQL, this is the equivalent)

--

=====

```
CREATE DATABASE IF NOT EXISTS wallet_db;
USE wallet_db;
```

Assessment
M5 - Database Management

```
DROP PROCEDURE IF EXISTS place_order;
```

```
DROP TABLE IF EXISTS orders;
```

```
DROP TABLE IF EXISTS customers;
```

```
CREATE TABLE customers (
```

```
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    name VARCHAR(100) NOT NULL,
```

```
    wallet_balance DECIMAL(10,2) NOT NULL DEFAULT 0
```

```
);
```

```
CREATE TABLE orders (
```

```
    order_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    customer_id INT NOT NULL,
```

```
    restaurant_id INT NOT NULL,
```

```
    amount DECIMAL(10,2) NOT NULL,
```

```
    order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
```

```
);
```

```
INSERT INTO customers (name, wallet_balance) VALUES
```

```
('Amit', 1000.00),
```

```
('Riya', 200.00);
```

```
DELIMITER $$
```

```
CREATE PROCEDURE place_order(
```

```
    IN p_customer_id INT,
```

```
    IN p_restaurant_id INT,
```

Assessment
M5 - Database Management

```
IN p_amount    DECIMAL(10,2)
)
BEGIN
    DECLARE v_balance DECIMAL(10,2) DEFAULT NULL;

    -- any unexpected SQL error: undo everything
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;

        SELECT 'FAILED: SQL error - transaction rolled back' AS result;
    END;

    START TRANSACTION;

    -- lock the customer's row so two orders cannot spend the same balance
    SELECT wallet_balance INTO v_balance
    FROM customers
    WHERE customer_id = p_customer_id
    FOR UPDATE;

    IF v_balance IS NULL THEN
        ROLLBACK;

        SELECT 'FAILED: customer not found' AS result;
    ELSEIF v_balance < p_amount THEN
        ROLLBACK;

        SELECT 'FAILED: insufficient wallet balance - rolled back' AS result;
    ELSE
        UPDATE customers
```

Assessment
M5 - Database Management

SET wallet_balance = wallet_balance - p_amount

WHERE customer_id = p_customer_id;

INSERT INTO orders (customer_id, restaurant_id, amount)

VALUES (p_customer_id, p_restaurant_id, p_amount);

COMMIT;

SELECT 'SUCCESS: order placed and wallet debited' AS result;

END IF;

END\$\$

DELIMITER ;

-- Test 1: valid order (Amit has 1000, order 350) -> SUCCESS

CALL place_order(1, 101, 350.00);

SELECT * FROM customers;

SELECT * FROM orders;

-- Test 2: order exceeds balance (Riya has 200, order 500) -> FAILED, nothing changes

CALL place_order(2, 101, 500.00);

SELECT * FROM customers;

SELECT * FROM orders;

--

=====

-- SECTION C - MINI CAPSTONE: Food Delivery Platform Backend

--

=====

DROP DATABASE IF EXISTS food_delivery_db;

Assessment
M5 - Database Management

```
CREATE DATABASE food_delivery_db;
```

```
USE food_delivery_db;
```

```
CREATE TABLE restaurants (  
    restaurant_id INT AUTO_INCREMENT PRIMARY KEY,  
    name    VARCHAR(100) NOT NULL,  
    city    VARCHAR(50) NOT NULL,  
    cuisine_type VARCHAR(50),  
    created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE menu_items (  
    item_id    INT AUTO_INCREMENT PRIMARY KEY,  
    restaurant_id INT NOT NULL,  
    item_name  VARCHAR(100) NOT NULL,  
    price     DECIMAL(8,2) NOT NULL,  
    category  VARCHAR(50) NOT NULL,  
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)  
);
```

```
CREATE TABLE orders (  
    order_id    INT AUTO_INCREMENT PRIMARY KEY,  
    customer_name VARCHAR(100) NOT NULL,  
    restaurant_id INT NOT NULL,  
    item_id     INT NOT NULL,  
    quantity   INT NOT NULL,  
    total_amount DECIMAL(10,2) NOT NULL,  
    order_date  DATE NOT NULL,
```

Assessment
M5 - Database Management

```
FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id),  
FOREIGN KEY (item_id) REFERENCES menu_items(item_id)  
);
```

```
CREATE TABLE order_audit (  
    audit_id INT AUTO_INCREMENT PRIMARY KEY,  
    order_id INT NOT NULL,  
    restaurant_id INT NOT NULL,  
    action VARCHAR(20) NOT NULL,  
    log_time DATETIME NOT NULL  
);
```

-- TRIGGER (created before the data load so every order is audited)

DELIMITER \$\$

CREATE TRIGGER after_order_insert

AFTER INSERT ON orders

FOR EACH ROW

BEGIN

INSERT INTO order_audit (order_id, restaurant_id, action, log_time)

VALUES (NEW.order_id, NEW.restaurant_id, 'INSERT', NOW());

END\$\$

DELIMITER ;

-- Data: 3 restaurants, 4 categories, 12 menu items, 15 orders

INSERT INTO restaurants (name, city, cuisine_type) VALUES

('Spice Villa', 'Rajkot', 'North Indian'),

('Pizza Palace', 'Ahmedabad', 'Italian'),

('Dragon Wok', 'Surat', 'Chinese');

Assessment
M5 - Database Management

```
INSERT INTO menu_items (restaurant_id, item_name, price, category) VALUES
```

```
(1, 'Paneer Tikka', 220.00, 'Starter'), -- item 1
(1, 'Dal Makhani', 260.00, 'Main Course'), -- item 2
(1, 'Gulab Jamun', 90.00, 'Dessert'), -- item 3
(1, 'Masala Chai', 40.00, 'Beverage'), -- item 4
(2, 'Garlic Bread', 150.00, 'Starter'), -- item 5
(2, 'Margherita Pizza', 320.00, 'Main Course'), -- item 6
(2, 'Tiramisu', 180.00, 'Dessert'), -- item 7
(2, 'Cold Coffee', 120.00, 'Beverage'), -- item 8
(3, 'Veg Spring Rolls', 170.00, 'Starter'), -- item 9
(3, 'Hakka Noodles', 240.00, 'Main Course'), -- item 10
(3, 'Ice Cream', 100.00, 'Dessert'), -- item 11
(3, 'Lemon Iced Tea', 90.00, 'Beverage'); -- item 12
```

```
-- total_amount = price * quantity
```

```
INSERT INTO orders (customer_name, restaurant_id, item_id, quantity, total_amount,
order_date) VALUES
```

```
('Amit', 1, 1, 2, 440.00, '2026-08-03'),
('Riya', 1, 2, 1, 260.00, '2026-08-10'),
('Karan', 1, 3, 3, 270.00, '2026-08-21'),
('Neha', 1, 4, 4, 160.00, '2026-09-02'),
('Amit', 1, 2, 2, 520.00, '2026-09-14'),
('Priya', 2, 5, 1, 150.00, '2026-08-05'),
('Rahul', 2, 6, 2, 640.00, '2026-08-19'),
('Sneha', 2, 7, 1, 180.00, '2026-09-01'),
('Karan', 2, 8, 3, 360.00, '2026-09-08'),
('Riya', 2, 6, 1, 320.00, '2026-09-16'),
```

Assessment
M5 - Database Management

```
('Vikram', 3, 9, 2, 340.00, '2026-08-12'),  
('Neha', 3, 10, 1, 240.00, '2026-08-25'),  
('Amit', 3, 11, 4, 400.00, '2026-09-04'),  
('Priya', 3, 12, 2, 180.00, '2026-09-11'),  
('Rahul', 3, 10, 3, 720.00, '2026-09-18');
```

-- VIEW: orders and revenue per restaurant

```
CREATE VIEW restaurant_sales_summary AS  
  
SELECT r.name           AS restaurant_name,  
       COUNT(o.order_id) AS total_orders,  
       COALESCE(SUM(o.total_amount), 0) AS total_revenue  
  
FROM restaurants r  
  
LEFT JOIN orders o ON o.restaurant_id = r.restaurant_id  
  
GROUP BY r.restaurant_id, r.name;
```

-- STORED PROCEDURE: add_order

```
DELIMITER $$  
  
CREATE PROCEDURE add_order(  
    IN p_customer_name VARCHAR(100),  
    IN p_restaurant_id INT,  
    IN p_item_id    INT,  
    IN p_quantity   INT  
)  
  
BEGIN  
  
    DECLARE v_exists INT DEFAULT 0;  
  
    DECLARE v_price  DECIMAL(8,2) DEFAULT NULL;  
  
  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

Assessment
M5 - Database Management

```
BEGIN

    ROLLBACK;

    SELECT 'FAILED: SQL error - rolled back' AS result;

END;


START TRANSACTION;


SELECT COUNT(*) INTO v_exists
FROM restaurants
WHERE restaurant_id = p_restaurant_id;


IF v_exists = 0 THEN

    ROLLBACK;

    SELECT 'FAILED: restaurant does not exist - rolled back' AS result;

ELSEIF p_quantity IS NULL OR p_quantity <= 0 THEN

    ROLLBACK;

    SELECT 'FAILED: quantity must be at least 1' AS result;

ELSE

    SELECT price INTO v_price
    FROM menu_items
    WHERE item_id = p_item_id AND restaurant_id = p_restaurant_id;


    IF v_price IS NULL THEN

        ROLLBACK;

        SELECT 'FAILED: item not found for this restaurant' AS result;

    ELSE

        INSERT INTO orders (customer_name, restaurant_id, item_id, quantity,
total_amount, order_date)
```

Assessment
M5 - Database Management

```
VALUES (p_customer_name, p_restaurant_id, p_item_id, p_quantity,
        v_price * p_quantity, CURDATE());

COMMIT;

SELECT 'SUCCESS: order added' AS result, LAST_INSERT_ID() AS new_order_id;

END IF;

END IF;

END$$

DELIMITER ;
```

-- Tests

```
SELECT * FROM restaurant_sales_summary;

CALL add_order('Meera', 2, 6, 2);  -- valid

CALL add_order('Meera', 99, 6, 2); -- invalid restaurant -> rollback

SELECT * FROM orders ORDER BY order_id DESC LIMIT 3;

SELECT * FROM order_audit;        -- 15 initial rows + 1 for Meera's order

SELECT * FROM restaurant_sales_summary;
```

--

=====

-- SECTION D - STEP 1: AI-built procedure (monthly restaurant summary)

-- Uses food_delivery_db.orders from Section C.

-- This is the "original AI version". Step 2 (find a bug/limitation and

-- fix it) you must do yourself after testing - see the notes in chat.

--

=====

USE food_delivery_db;

DROP PROCEDURE IF EXISTS get_monthly_summary;

Assessment
M5 - Database Management

```
DELIMITER $$

CREATE PROCEDURE get_monthly_summary(
    IN p_restaurant_id INT,
    IN p_month INT
)
BEGIN
    DECLARE v_count INT DEFAULT 0;

    SELECT COUNT(*) INTO v_count
    FROM orders
    WHERE restaurant_id = p_restaurant_id
    AND MONTH(order_date) = p_month;

    IF v_count = 0 THEN
        SELECT CONCAT('No orders found for restaurant ', p_restaurant_id,
            ' in month ', p_month) AS message;
    ELSE
        SELECT restaurant_id,
            COUNT(*) AS total_orders,
            SUM(total_amount) AS total_revenue,
            ROUND(AVG(total_amount), 2) AS avg_order_value
        FROM orders
        WHERE restaurant_id = p_restaurant_id
        AND MONTH(order_date) = p_month
        GROUP BY restaurant_id;
    END IF;
END$$

DELIMITER ;
```

Assessment
M5 - Database Management

CALL get_monthly_summary(1, 8); -- has data

CALL get_monthly_summary(2, 9); -- has data

CALL get_monthly_summary(3, 1); -- no orders -> message

Assessment

M5 - Database Management

The image displays two screenshots of the phpMyAdmin web interface, showing the 'Structure' tab for two different databases: 'foodapp_db' and 'menu_db'.

Top Screenshot: foodapp_db

The interface shows the 'Structure' tab for the 'foodapp_db' database. The left sidebar lists various databases, including 'foodapp_db', 'hello_db', 'information_schema', 'instacore', 'menu_db', 'mydb', 'mysql', 'performance_schema', 'phpmyadmin', 'product', 'test', and 'wallet_db'. The main panel displays a table list with the following data:

Table	Action	Rows	Type	Collation	Size	Overhead
orders	Browse Structure Search Insert Empty Drop	14	InnoDB	utf8mb4_general_ci	32.0 KiB	-
restaurants	Browse Structure Search Insert Empty Drop	6	InnoDB	utf8mb4_general_ci	16.0 KiB	-
2 tables	Sum	20	InnoDB	utf8mb4_general_ci	48.0 KiB	0 B

Below the table list, there is a 'Create new table' section with fields for 'Table name' and 'Number of columns' (set to 4), and a 'Create' button.

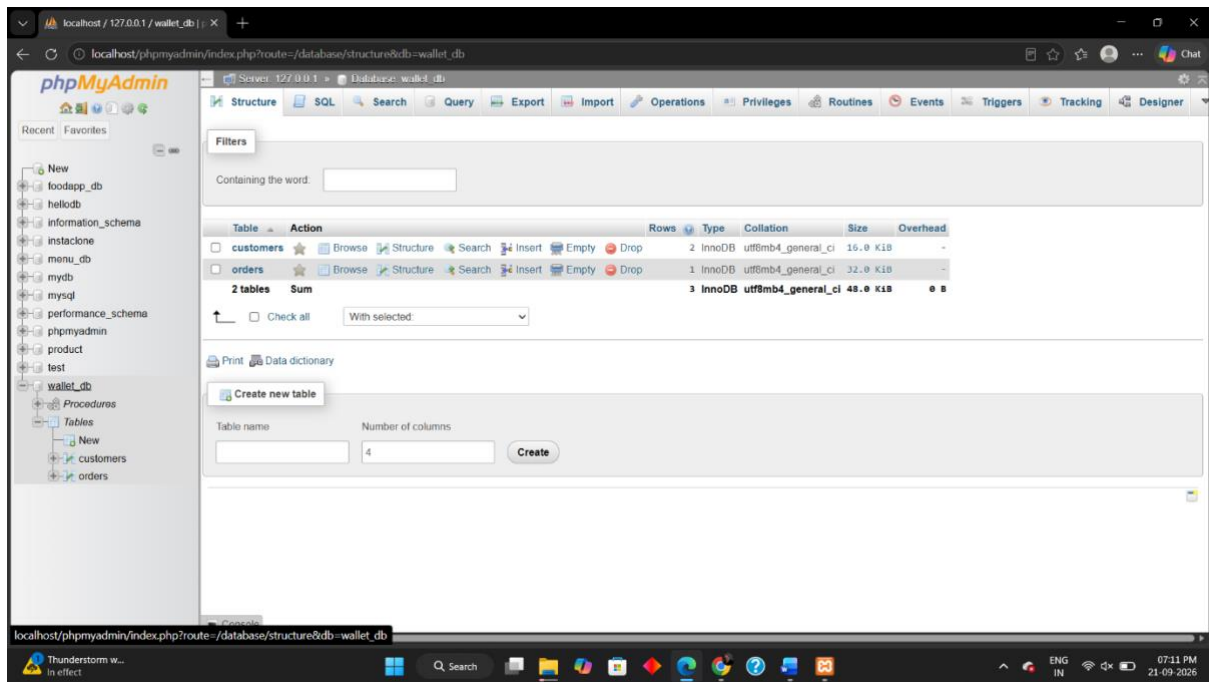
Bottom Screenshot: menu_db

The interface shows the 'Structure' tab for the 'menu_db' database. The left sidebar lists various databases, including 'foodapp_db', 'hello_db', 'information_schema', 'instacore', 'menu_db', 'mydb', 'mysql', 'performance_schema', 'phpmyadmin', 'product', 'test', and 'wallet_db'. The main panel displays a table list with the following data:

Table	Action	Rows	Type	Collation	Size	Overhead
menu_items	Browse Structure Search Insert Empty Drop	12	InnoDB	utf8mb4_general_ci	32.0 KiB	-
restaurants	Browse Structure Search Insert Empty Drop	5	InnoDB	utf8mb4_general_ci	16.0 KiB	-
restaurant_menu_summary	Browse Structure Search Insert Edit Drop	~0	View	-	-	-
3 tables	Sum	~17	InnoDB	utf8mb4_general_ci	48.0 KiB	0 B

Below the table list, there is a 'Create new table' section with fields for 'Table name' and 'Number of columns' (set to 4), and a 'Create' button.

Assessment M5 - Database Management



The script starts Section C with DROP DATABASE IF EXISTS food_delivery_db;, which deletes any existing database with that name.